PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO

Master em Inteligência Artificial Generativa & Large Language Models

DataBrief AI: um agente conversacional multimodal para refinamento de demandas de dados

Philipe da Silva Simões

Trabalho apresentado ao curso GenAI e LLM Master como pré-requisito para conclusão de curso e obtenção de crédito na disciplina "Projeto Final ".

Orientador: Prof. Leonardo Alfredo Forero Mendoza

Matrícula: 252.100.219 · Turma: 2025-2

Rio de Janeiro, outubro de 2026

DataBrief AI: um agente conversacional multimodal para refinamento de demandas de dados

Resumo

Este trabalho apresenta o DataBrief AI, um agente conversacional que transforma pedidos vagos de dados em briefings estruturados, revisados e aprovados por um humano. O agente aceita texto, áudio e anexos, roda no Google Colab gratuito e oferece três modos de execução: um modelo local de 4 bilhões de parâmetros em GPU, um modelo local menor em CPU e uma API comercial.

A arquitetura combina um grafo de estados (LangGraph), extração de campos por modelo de linguagem, guardas determinísticas contra alucinação e um registro de proveniência por campo. Avaliado em 10 casos sintéticos, o agente atingiu todas as metas de processo nos dois modos avaliados por completo: recall de lacunas de 1,00, nenhuma pergunta repetida ou desnecessária e completude de 1,00.

A concordância humana foi de 0,815 com gpt-4o-mini e de 0,704 com Qwen3-4B local, abaixo da meta de 0,80. A diferença se concentra em títulos gerados de forma vaga pelo modelo menor, o que motivou tornar o título editável antes da aprovação.

Palavras-chave: agentes conversacionais; modelos de linguagem pequenos; refinamento de requisitos; proveniência; humano no circuito.

1. Problema

Em times de dados corporativos, a entrada da demanda é um gargalo pouco visível. O pedido chega por mensagem, áudio ou e-mail encaminhado e raramente diz o formato de entrega, a pergunta de negócio que deve responder ou se a necessidade é pontual ou contínua.

O contexto deste trabalho é a Gerência de Inteligência de Dados da YDUQS, holding de ensino superior com marcas como Estácio, Wyden e Ibmec. A gerência registra demandas numa lista padronizada do SharePoint, com campos como tipo de demanda, valor de negócio, classificação estratégica e perguntas de negócio. Preencher essa lista exige, em geral, uma reunião de descoberta com o solicitante.

Persona. Coordenador ou analista sênior de dados que recebe demandas de áreas de negócio e precisa registrá-las nesse formato. O usuário domina a taxonomia; não há usuário leigo.

Hipótese de valor. Um agente que pergunta apenas o que falta, na ordem certa, e mostra de onde veio cada informação reduz o caminho entre o pedido e um briefing aprovado, sem retirar do humano a decisão final.

Restrições de projeto. Execução no Google Colab gratuito, onde a GPU T4 não é garantida e tem cerca de 15 GB de memória; uso preferencial de modelos abertos e locais; aprovação humana obrigatória antes de qualquer saída; nenhum envio automático.

2. Solução

O DataBrief AI conduz uma conversa curta que termina num briefing com sete campos obrigatórios, cada um com a origem registrada. Uma sessão típica segue cinco passos:

1. O usuário descreve a demanda por texto, áudio ou anexo (PDF, DOCX ou TXT).
2. Áudio e anexo passam por uma tela de revisão: o texto transcrito ou extraído fica editável antes de entrar na conversa, o que permite corrigir uma sigla mal reconhecida.
3. O agente extrai os campos que conseguir e pergunta apenas o que falta, um campo por vez. Campos de opções fixas aparecem como botões.
4. Completo o estado, o agente gera título e resumo e exibe o briefing com um selo de origem por campo: digitado, áudio, anexo, inferido pelo sistema ou selecionado.
5. O usuário pode editar título e objetivo e então aprova. Só depois disso o JSON fica disponível para download e o resumo em áudio pode ser gerado.

Campo	Forma de resolução
Tipo de demanda (Análise, Produto de Dados, Alarmística)	Extraído do texto ou escolhido em botões
Objetivo	Extraído do texto livre
Resultado esperado	Regra para Análise e Alarmística; botões para Produto de Dados
Valor de negócio (Operacional, Tático, Estratégico)	Sempre escolhido em botões
Classificação estratégica (até 3 de 8)	Extraída ou escolhida em caixas de seleção
Perguntas de negócio	Sugeridas pelo modelo, editadas e confirmadas pelo usuário
Título	Gerado pelo sistema, editável antes de aprovar

O princípio que orienta a divisão de trabalho é simples: o modelo de linguagem trata o que só ele faz bem, que é interpretar texto livre, e regras determinísticas ou escolhas do usuário tratam campos de opções fechadas. A aprovação é sempre humana e o envio, sempre manual.

3. Arquitetura

O agente é um grafo de estados com quatro nós, cercado por etapas de entrada e saída em que o humano revisa antes de seguir.

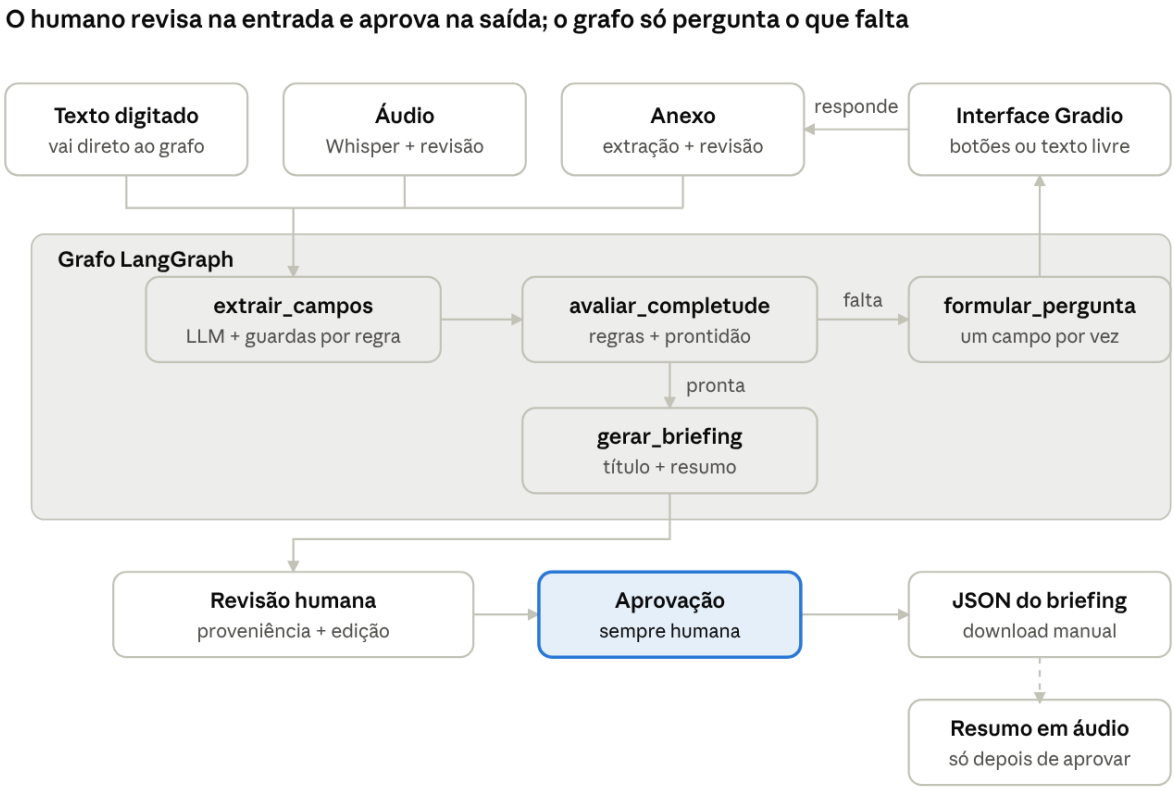


Figura 1 · fluxo do DataBrief AI, da entrada ao download

A cada resposta, o usuário volta pelo topo; o grafo só sai do ciclo quando não falta nenhum campo, e nada sai do sistema sem a aprovação.

3.1 Grafo e estado

O grafo, implementado em LangGraph, tem os nós extrair_campos, avaliar_completude, formular_pergunta e gerar_briefing. Depois da extração, a avaliação decide se ainda faltam campos (vai para a pergunta) ou se a demanda está pronta (vai para o briefing). O ciclo se repete a cada turno do usuário.

Todo o estado vive em objetos Pydantic: uma sessão contém demandas, e cada demanda guarda campos, histórico de turnos, contagem de tentativas por campo e latência por etapa. A interface serializa esse estado em JSON entre um evento e outro e o reconstrói a cada clique, sem variáveis globais.

3.2 Proveniência

Cada campo de texto livre é armazenado com valor, origem (TEXT, AUDIO, ATTACHMENT, RULE ou MANUAL), turno, arquivo de origem e um indicador de confirmação. Os três campos de opção fixa mantêm o valor cru, porque várias regras comparam esses valores diretamente, e registram a origem

num dicionário paralelo. Na aprovação, todos os campos são marcados como confirmados e exportados no JSON.

3.3 Modos de execução e carga sequencial

Modo	Modelo de linguagem	Reconhecimento	Papel
GPU_LOCAL	Qwen3-4B-Instruct, quantizado em 4 bits	Whisper Small	Principal e de demonstração
CPU_LOCAL	Qwen3-1.7B, bf16	Whisper Tiny	Contingência sem GPU
OPENAI	gpt-4o-mini via API	Whisper local	Alternativo, com chave institucional

O modo é registrado uma única vez, na inicialização, e todas as chamadas ao modelo passam por uma função única que despacha conforme esse registro. O sistema nunca troca de modo sozinho: se faltar GPU ou chave, ele para com uma mensagem clara. Dois defeitos reais violavam essa regra e foram corrigidos durante o projeto, um seletor de modo que só alterava um metadado e um valor padrão que mascarava a ausência do parâmetro de modo.

Como Whisper e Qwen não cabem juntos com folga na T4, a carga é sequencial: o Whisper carrega, transcreve e libera a memória antes de o Qwen voltar a responder. O sintetizador de voz (Piper) roda em CPU e só é carregado depois da aprovação.

3.4 Guardas contra alucinação

Testes ao vivo mostraram que o modelo local preenchia campos sem base no texto. Cada guarda confere o texto original do usuário, não a resposta do modelo:

- **Resultado esperado** só é aceito se o texto contiver uma palavra de formato, como dashboard ou alerta.
- **Tipo de demanda** não é gravado quando o texto aciona palavras de duas categorias ao mesmo tempo, nem quando o turno é só uma frase de transmissão ("segue o anexo").
- **Título** só é aceito em texto livre se a pergunta anterior era de fato sobre título; no caminho normal, é gerado a partir do estado já confirmado.
- **Valor de negócio** deixou de ser extraído: o modelo errou 3 de 3 vezes nos dois tamanhos, confundindo Tático com Operacional quando o texto falava em eficiência operacional.

3.5 Decisões de escopo

Recuperação aumentada (RAG) testada e removida. Um pipeline com embeddings E5-small e índice FAISS foi implementado e testado com um corpus real da YDUQS: 54 trechos, sendo 28 páginas da apresentação corporativa e 26 seções do site institucional. A intenção era que o agente conhecesse melhor a empresa para fazer perguntas mais específicas ao solicitante.

Na prática, o modelo recuperava os cinco trechos mais parecidos com a demanda e os citava no briefing, em vez de usá-los para perguntar melhor. Em três demandas de teste, uma recebeu uma citação útil;

outra, trechos quase repetidos de uma mesma página de gráfico; e a terceira, sobre consumo de energia de data centers, recebeu quatro trechos sem relação com o pedido, apresentados como relevantes.

Por que foi removido. Com a stack do projeto, o RAG não funcionou bem: um modelo local de 4 bilhões de parâmetros, decidindo sozinho o que era relevante, não teve capacidade para separar o trecho útil do irrelevante. Ao mesmo tempo, o agente já entregava briefings completos sem ele, porque o que falta numa demanda vaga vem das perguntas ao solicitante. Mantê-lo só acrescentaria risco de ruído no briefing, então seguimos sem ele. O código e os testes ficaram preservados no histórico do repositório.

Três tipos de demanda, não quatro. A categoria Estruturante (pipelines, tabelas Gold) sempre servia de base para um produto ou um alerta, nunca era o pedido final de uma área de negócio. Passou a existir só como classificação estratégica.

4. Avaliação

Os dois modos avaliados por completo atingiram todas as metas de processo; só a concordância humana do modelo local ficou abaixo da meta.

4.1 Conjunto de dados

Foram usados 10 casos sintéticos, um por categoria de situação: tema sem formato, formato explícito, bloqueio com link, alerta explícito, pergunta de negócio, caso fechado, anexo, ambiguidade entre categorias, entrada por áudio e demanda totalmente vaga. As demandas são inventadas, mas o vocabulário vem da YDUQS (marcas, modalidades, captação, evasão, renovação). Cada caso traz respostas previstas por campo e um gabarito final.

4.2 Método

Um script percorre cada caso até o briefing ficar pronto, simulando as respostas do usuário a partir das respostas previstas, e calcula as métricas automáticas. F1 é calculado por campo categórico; para a classificação estratégica, sobre a interseção dos conjuntos. Os campos de texto livre (título, objetivo e perguntas de negócio) geram 27 itens por modo, julgados por um avaliador humano como "concordo" ou "discordo" em relação ao gabarito.

4.3 Resultados de qualidade

Métrica	Meta	GPU_LOCAL (Qwen3-4B)	OPENAI (gpt-4o-mini)
F1 tipo de demanda	—	0,90	1,00
F1 valor de negócio	—	1,00	1,00
F1 classificação estratégica	—	0,817	0,967
F1 resultado esperado	—	0,90	1,00
Recall de lacunas	≥ 0,90	1,00	1,00
Perguntas repetidas	0	0	0

Métrica	Meta	GPU_LOCAL (Qwen3-4B)	OPENAI (gpt-4o-mini)
Perguntas desnecessárias	≤ 15%	0%	0%
Turnos até pronta (média)	—	3,4	4,1
Completude média	≥ 0,85	1,00	1,00
Concordância humana	≥ 0,80	0,704 (19 de 27)	0,815 (22 de 27)

Rodadas de 07/09/2026 (GPU_LOCAL) e 12/09/2026 (OPENAI). Na primeira rodada de GPU_LOCAL, um caso não chegou a pronto e a completude média foi 0,943; o caso de teste foi ajustado e a segunda rodada fechou os 10 casos.

4.4 Resultados de latência

Modo	Lote de 10 casos	Extração P50 / P95	Briefing P50
CPU_LOCAL	63,2 min	247,5 s / 276,0 s	190,4 s (1 caso)
GPU_LOCAL	3,6 min	8,3 s / 16,7 s	22,3 s
OPENAI	1,0 min	1,0 s / 1,3 s	2,1 s

O modo OPENAI usou 144 chamadas e cerca de 62 mil tokens para os 10 casos. O CPU_LOCAL tem só medição de latência; a avaliação de qualidade não foi feita nesse modo.

4.5 Análise

O gpt-4o-mini bate todas as metas. O Qwen3-4B bate as metas automáticas, e a diferença está em dois pontos. No F1 de tipo de demanda e resultado esperado, um único caso (a demanda totalmente vaga) explica a perda: diante de um texto quase vazio, o modelo arriscou uma categoria. Na concordância humana, a maior parte das discordâncias está nos títulos dos casos C006 a C010: o modelo resume demais os recortes que o usuário mencionou, por exemplo omitindo a praça ou a marca. Esse padrão já aparecia nas primeiras rodadas ao vivo, antes da avaliação formal.

As metas de processo (nenhuma pergunta repetida, nenhuma desnecessária, nenhuma lacuna esquecida) dependem sobretudo da lógica determinística do grafo e foram iguais nos dois modos. Isso sugere que a arquitetura isola o comportamento da conversa da qualidade do modelo, e que a escolha do modelo pesa principalmente nos campos de texto livre.

4.6 Recuperação de falhas

Seis cenários de erro deliberado foram catalogados: anexo corrompido, áudio sem fala, chave da API ausente ou inválida, modo passado com tipo errado, GPU indisponível e biblioteca de quantização ausente. Os dois primeiros eram defeitos reais, sem nenhum tratamento, e foram corrigidos. Os seis terminam com mensagem clara ao usuário. Não foi calculada uma taxa formal, porque dois cenários existem só no notebook e foram validados manualmente.

5. Limitações

Os números acima valem para um protótipo acadêmico e devem ser lidos com as seguintes ressalvas.

- **Um único avaliador humano**, o próprio autor, sem rubrica calibrada nem medida de concordância entre avaliadores.
- **Avaliação de qualidade do CPU_LOCAL.** Devido ao tempo disponível, a avaliação de qualidade não foi feita nesse modo: um ciclo completo levaria de 60 a 90 minutos de execução, mais a revisão humana. Isso não afeta as conclusões, porque o CPU_LOCAL é a contingência para quando não há GPU. A qualidade foi medida nos dois modos de uso principal, e o CPU_LOCAL rodou os 10 casos de ponta a ponta, sem erro, com a latência medida.
- **Completude influenciada pelo método.** O script fornece resposta para todo campo perguntado, então todo caso chega a pronto. A métrica mostra que o fluxo fecha, não que um usuário real responderia tudo.
- **Variabilidade do modelo local.** O mesmo caso fechou em 3 turnos numa rodada e travou no limite de 8 turnos em outra; a geração não é determinística.
- **Campos específicos por tipo ausentes.** Uma Alarmística não registra limite nem frequência de alerta; uma Análise não registra recorte temporal. O schema previsto nunca foi ligado ao grafo e foi removido.
- **Uma demanda por sessão.** O desmembramento de pedidos compostos foi desenhado e não ativado.
- **Sem persistência.** O briefing existe fora da sessão do Colab só se o usuário fizer o download.
- **Exposição do link.** A interface do Gradio no Colab gera um link público temporário, sem autenticação; há um aviso na tela.

6. Trabalhos futuros

Em ordem de prioridade, com a justificativa de cada item:

1. **Ampliar para 50 casos e incluir um segundo avaliador.** É o que mais aumenta a confiança nos resultados; hoje cada caso pesa 10% de cada métrica.
2. **Corpus interno da YDUQS.** O corpus público não ajudou, mas um corpus interno (catálogo de tabelas Gold, dashboards e alertas existentes) pode evitar demandas duplicadas. A busca seria acionada só em demandas não sensíveis.
3. **RAG agêntico.** Em vez de recuperar a cada turno, o próprio agente decidiria se, quando e onde buscar. O grafo atual comporta isso como um nó novo com aresta condicional, sem reescrever os existentes.
4. **Campos específicos por tipo**, começando pela Alarmística, em que um briefing incompleto custa mais.
5. **Avaliação de qualidade do modo CPU_LOCAL**, para saber se a contingência é utilizável.
6. **Rubrica calibrada e exemplos reais anonimizados** como referência para o modelo e para o avaliador.
7. **Fluxo de múltiplas demandas**, com ordenação por dependência técnica.
8. **Persistência, hospedagem permanente, autenticação e integração** com SharePoint, Jira ou Azure DevOps, depois que os itens anteriores mostrarem valor em uso real.

7. Conclusão

O DataBrief AI mostra que um agente com modelo local de 4 bilhões de parâmetros, rodando em infraestrutura gratuita, consegue conduzir o refinamento de uma demanda de dados até um briefing completo, sem perguntas repetidas ou desnecessárias.

O principal aprendizado é de arquitetura. Confiar no modelo para tudo produziu campos inventados em testes reais; a confiabilidade veio de restringir o modelo ao texto livre e cercá-lo de regras determinísticas, escolhas explícitas do usuário e um registro de proveniência que deixa cada inferência visível antes da aprovação.

A diferença de qualidade entre o modelo local e a API comercial ficou concentrada nos campos de texto livre, sobretudo no título. A resposta adotada foi tornar o título editável e marcar a edição como manual, em vez de esconder o limite do modelo. Com um conjunto maior de casos e um corpus interno, a ferramenta pode passar de prova de conceito a apoio real à entrada de demandas da gerência.

Referências

ABID, A. et al. Gradio: hassle-free sharing and testing of ML models in the wild. arXiv:1906.02569, 2019.

DETTMERS, T. et al. QLoRA: efficient finetuning of quantized LLMs. In: Advances in Neural Information Processing Systems, 2023. arXiv:2305.14314.

LANGCHAIN. LangGraph documentation. Disponível em: langchain-ai.github.io/langgraph. Acesso em: out. 2026.

OPENAI. GPT-4o mini. Disponível em: platform.openai.com/docs. Acesso em: set. 2026.

PYDANTIC. Pydantic documentation, v2. Disponível em: docs.pydantic.dev. Acesso em: out. 2026.

QWEN TEAM. Qwen3 technical report. arXiv:2505.09388, 2025.

RADFORD, A. et al. Robust speech recognition via large-scale weak supervision. arXiv:2212.04356, 2022.

RHASSPY. Piper: a fast, local neural text to speech system. Disponível em: github.com/rhasspy/piper. Acesso em: ago. 2026.

SIMÕES, P. S. DataBrief AI: repositório do projeto. Disponível em: github.com/philsimoes/databrief-ai. Acesso em: out. 2026.

WANG, L. et al. Multilingual E5 text embeddings: a technical report. arXiv:2402.05672, 2024.

YDUQS. Apresentação corporativa. Rio de Janeiro: YDUQS, jun. 2026.